JavaScript Snippets Vault

150+ Essential Code Snippets for Modern Developers

52 Premium Snippets

A curated collection of production-ready JavaScript utilities for frontend engineers, backend developers, and full-stack builders.

All snippets are self-contained, dependency-free, and compatible with modern browsers and Node.js 18+.

Created by

DevFlow Suite

v1.0 • August 2025

Introduction

Welcome to the JavaScript Snippets Vault — a curated collection of 52 production-ready JavaScript utilities organized into 7 categories. Each snippet has been carefully written to be concise, efficient, and battle-tested in real-world applications.

What you will learn:

• 8 DOM manipulation utilities for efficient UI interactions

• 8 async patterns for robust promise-based code

• 14 array and object methods for data transformation

• 6 string utilities for text processing

• 8 browser API helpers for modern web features

• 4 CSS and layout helpers for responsive design

• 4 performance and debugging tools

All snippets are 100% client-side, require no external dependencies, and can be copied directly into your project. Happy coding!

Table of Contents

Browser APIs

CSS & Layout

Performance & Debugging

DOM Manipulation

#1 Select Elements

```
// Single element
const el = document.querySelector('.btn-primary');

// Multiple elements
const items = document.querySelectorAll('.card');

// By ID (fastest)
const header = document.getElementById('header');
```

Use querySelector for a single element, querySelectorAll for multiple.

#2 Create & Append Element

```
const div = document.createElement('div');
div.className = 'new-element';
div.textContent = 'Hello World';

document.body.appendChild(div);
// Or: document.body.append(div);
```

Create new DOM nodes and insert them into the document tree.

#3 Remove Element

```
const el = document.querySelector('.to-remove');
if (el) {
el.remove();
// Fallback: el.parentNode.removeChild(el);
}
```

Remove an element from the DOM with proper cleanup.

#4 Get/Set Attributes

```
const link = document.querySelector('a');
const url = link.getAttribute('href');
link.setAttribute('href', 'https://devflow.suite');
link.setAttribute('target', '_blank');
link.removeAttribute('target');
```

Manipulate HTML attributes using the element API.

#5 Toggle CSS Classes

```
const menu = document.querySelector('.menu');
menu.classList.toggle('open');
menu.classList.toggle('hidden', !menu.classList.contains('open'));
const isOpen = menu.classList.contains('open');
```

Add, remove, or toggle CSS classes dynamically.

#6 Event Delegation

```
document.getElementById('list').addEventListener('click', (e) => {
if (e.target.matches('li')) {
console.log('Clicked:', e.target.textContent);
}
});
```

Single event listener on parent for child elements.

#7 Detect Click Outside

```
function clickOutside(el, cb) {
const handler = (e) => {
if (!el.contains(e.target)) cb(e);
};
document.addEventListener('click', handler);
return () => document.removeEventListener('click', handler);
}
```

Close dropdowns or modals when clicking outside the element.

#8 Smooth Scroll to Element

```
function scrollToEl(selector) {
document.querySelector(selector)?.scrollIntoView({
behavior: 'smooth',
block: 'start'
});
}
```

Scroll to a specific element with smooth animation.

Async & Promises

#9 Sleep / Delay

```
const sleep = (ms) => new Promise(r => setTimeout(r, ms));

// Usage
await sleep(1000);
console.log('1 second passed');
```

Pause execution for a specified duration using promises.

#10 Debounce Function

```
function debounce(fn, delay) {
let timer;
return (...args) => {
clearTimeout(timer);
timer = setTimeout(() => fn.apply(this, args), delay);
};
}

// Usage
document.querySelector('#search').addEventListener('input',
debounce(e => searchAPI(e.target.value), 300));
```

Delay function execution until after a period of inactivity.

#11 Throttle Function

```
function throttle(fn, limit) {
let inThrottle;
return (...args) => {
if (!inThrottle) {
fn.apply(this, args);
inThrottle = true;
setTimeout(() => inThrottle = false, limit);
}
};
}
```

Ensure a function is only called once per time period.

#12 Retry Failed Request

```
async function retry(fn, retries = 3, delay = 1000) {
try {
return await fn();
} catch (err) {
if (retries <= 0) throw err;
await sleep(delay);
return retry(fn, retries - 1, delay);
}
}

// Usage
const data = await retry(() => fetch('/api/data'));
```

Automatically retry a promise on failure with optional delay.

#13 Promise with Timeout

```
function withTimeout(promise, ms) {
const timeout = new Promise((_, reject) =>
setTimeout(() => reject(new Error('Timeout after ' + ms + 'ms')), ms)
);
return Promise.race([promise, timeout]);
}

// Usage
const data = await withTimeout(fetch('/api'), 5000);
```

Race a promise against a timeout that rejects on expiry.

#14 Concurrency-Limited Promise.All

```
async function promisePool(tasks, concurrency = 3) {
const results = [];
for (let i = 0; i < tasks.length; i += concurrency) {
const batch = tasks.slice(i, i + concurrency);
results.push(...await Promise.all(batch.map(t => t())));
}
return results;
}

// Usage
await promisePool(urls.map(u => () => fetch(u)), 3);
```

Run async tasks in parallel with a configurable concurrency limit.

#15 Memoize Async Function

```
function memoizeAsync(fn) {
const cache = new Map();
return async function(...args) {
const key = JSON.stringify(args);
if (cache.has(key)) return cache.get(key);
const result = await fn.apply(this, args);
cache.set(key, result);
return result;
};
}
```

Cache results of expensive async operations by argument.

#16 Simple Event Emitter

```
class Emitter {
constructor() { this.events = {}; }
on(e, cb) { (this.events[e] ||= []).push(cb); return this; }
emit(e, ...args) { this.events[e]?.forEach(cb => cb(...args)); return this; }
off(e, cb) { if (e) this.events[e] = this.events[e]?.filter(f => f !== cb); return this; }
}

const emitter = new Emitter();
emitter.on('data', d => console.log(d));
```

Lighter alternative to Node.js EventEmitter for browsers.

Arrays & Objects

#17 Deep Clone

```
// Method 1: JSON (fastest, no functions)
const clone = JSON.parse(JSON.stringify(obj));

// Method 2: StructuredClone (modern browsers + Node 17+)
const clone2 = structuredClone(obj);

// Method 3: Recursive (handles functions)
function deepClone(obj, seen = new WeakMap()) {
if (obj === null || typeof obj !== 'object') return obj;
if (seen.has(obj)) return seen.get(obj);
const clone = Array.isArray(obj) ? [] : {};
seen.set(obj, clone);
for (const k in obj) clone[k] = deepClone(obj[k], seen);
return clone;
}
```

Create a deep copy of any value without shared references.

#18 Deep Merge

```
function deepMerge(target, ...sources) {
if (!sources.length) return target;
const src = sources.shift();
if (!isObject(target) || !isObject(src)) return target;
for (const k in src) {
if (isObject(src[k])) {
if (!target[k]) target[k] = {};
deepMerge(target[k], src[k]);
} else {
target[k] = src[k];
}
}
return target;
}
function isObject(x) { return x && typeof x === 'object'; }
```

Recursively merge two or more objects together.

#19 Group By Property

```
const groupBy = (arr, key) =>
arr.reduce((acc, item) => {
(acc[item[key]] ||= []).push(item);
return acc;
}, {});

// Usage
groupBy(users, 'role');
// { admin: [...], user: [...] }
```

Transform a flat array into an object grouped by a key.

#20 Unique / Deduplicate

```
// Primitives
const uniq = [...new Set(arr)];

// Objects by key
const uniqBy = (arr, key) =>
[...arr.reduce((m, item) => m.set(item[key], item), new Map()).values()];

uniq([1, 2, 2, 3]); // [1, 2, 3]
```

Remove duplicate values from an array.

#21 Flatten Nested Array

```
// Modern: .flat(Infinity)
const flat = arr.flat(Infinity);

// Recursive
const flatten = arr =>
arr.reduce((acc, v) =>
Array.isArray(v) ? acc.concat(flatten(v)) : acc.concat(v), []);

flatten([1, [2, [3, [4]]]]); // [1, 2, 3, 4]
```

Flatten arrays of arbitrary depth.

#22 Chunk Array

```
const chunk = (arr, size) => {
const chunks = [];
for (let i = 0; i < arr.length; i += size) {
chunks.push(arr.slice(i, i + size));
}
return chunks;
};

chunk([1,2,3,4,5,6,7], 3);
// [[1,2,3], [4,5,6], [7]]
```

Split an array into smaller arrays of a specified size.

#23 Array Intersection

```
const intersect = (a, ...arrays) =>
arrays.reduce((acc, arr) =>
acc.filter(x => arr.includes(x)), a);

intersect([1,2,3,4], [2,3,5], [3,4,2]); // [2, 3]
```

Return elements that exist in ALL provided arrays.

#24 Array Difference

```
const difference = (arr, ...arrays) => {
const exclude = new Set(arrays.flat());
return arr.filter(x => !exclude.has(x));
};

difference([1,2,3,4,5], [2,4], [5]); // [1, 3]
```

Return elements from first array not in any other arrays.

#25 Shuffle (Fisher-Yates)

```
function shuffle(arr) {
const a = [...arr];
for (let i = a.length - 1; i > 0; i--) {
const j = Math.floor(Math.random() * (i + 1));
[a[i], a[j]] = [a[j], a[i]];
}
return a;
}

shuffle([1,2,3,4,5]); // [3,1,5,2,4]
```

Randomly shuffle array elements without mutation.

#26 Random Array Element

```
const sample = arr => arr[Math.floor(Math.random() * arr.length)];

const sampleSize = (arr, n = 1) =>
Array.from({ length: n }, () => sample(arr));

sample(['a','b','c']); // 'b'
sampleSize([1,2,3,4,5], 3); // [4,1,5]
```

Return one or more random elements from an array.

#27 Sort By Property

```
const sortBy = (arr, key, dir = 'asc') => {
return [...arr].sort((a, b) => {
if (a[key] > b[key]) return dir === 'asc' ? 1 : -1;
if (a[key] < b[key]) return dir === 'asc' ? -1 : 1;
return 0;
});
};

sortBy(users, 'age', 'desc');
```

Sort an array of objects by a specific property.

#28 Sum / Average / Median

```
const sum = arr => arr.reduce((a, b) => a + b, 0);
const avg = arr => sum(arr) / arr.length;
const median = arr => {
const s = [...arr].sort((a,b) => a - b);
const mid = Math.floor(s.length / 2);
return s.length % 2 ? s[mid] : (s[mid-1] + s[mid]) / 2;
};

sum([1,2,3,4]); // 10
avg([1,2,3,4]); // 2.5
median([1,3,2,4]); // 2.5
```

Statistical calculations on numeric arrays.

#29 Range Generator

```
const range = (start, end, step = 1) => {
const result = [];
if (end === undefined) { end = start; start = 0; }
for (let i = start; i < end; i += step) result.push(i);
return result;
};

range(5); // [0,1,2,3,4]
range(1, 6); // [1,2,3,4,5]
range(0, 10, 2); // [0,2,4,6,8]
```

Generate an array of numbers within a range.

#30 Deep Equality Check

```
function deepEqual(a, b) {
if (a === b) return true;
if (typeof a !== typeof b) return false;
if (a == null || b == null) return a === b;
if (typeof a !== 'object') return a === b;
if (Array.isArray(a) !== Array.isArray(b)) return false;
const ka = Object.keys(a), kb = Object.keys(b);
if (ka.length !== kb.length) return false;
return ka.every(k => deepEqual(a[k], b[k]));
}
```

Check if two values are deeply equal.

#31 Get / Set Nested Property

```
// Get
const get = (obj, path, def) =>
path.split('.').reduce((o, k) =>
o?.[k] !== undefined ? o[k] : undefined, obj) ?? def;

// Set
const set = (obj, path, val) => {
const keys = path.split('.');
let cur = obj;
for (let i = 0; i < keys.length - 1; i++) {
cur = cur[keys[i]] = cur[keys[i]] || {};
}
cur[keys.at(-1)] = val;
return obj;
}

const obj = {a:{b:{c:1}}};
get(obj,'a.b.c'); // 1
set(obj,'a.b.d',2);
```

Access and modify nested object properties via dot-notation path.

#32 Pick / Omit Keys

```
const pick = (obj, ...keys) =>
Object.fromEntries(keys.map(k => [k, obj[k]]));

const omit = (obj, ...keys) =>
Object.fromEntries(Object.entries(obj).filter(([k]) => !keys.includes(k)));

// Usage
pick({a:1,b:2,c:3}, 'a','c'); // {a:1, c:3}
omit({a:1,b:2,c:3}, 'b'); // {a:1, c:3}
```

Select or remove specific keys from an object.

#33 Map & Group

```
function mapGroup(arr, mapFn, groupFn) {
const groups = {};
for (const item of arr) {
const mapped = mapFn(item);
const key = groupFn(item);
(groups[key] ||= []).push(mapped);
}
return groups;
}

// Usage: map and group users by role
mapGroup(users, u => u.name, u => u.role);
```

Apply a function and group results.

Strings & Utilities

#34 Slugify

```
function slugify(str) {
return str
.toLowerCase()
.trim()
.replace(/[^\w\s-]/g, '')
.replace(/[\s_-]+/g, '-')
.replace(/^-+|-+$/g, '');
}

slugify('Hello World!'); // 'hello-world'
```

Convert any string into a URL-friendly slug.

#35 Capitalize / Title Case

```
const capitalize = s => s.charAt(0).toUpperCase() + s.slice(1);

const titleCase = s =>
s.toLowerCase().replace(/(?:^|\s)\w/g, c => c.toUpperCase());

capitalize('hello'); // 'Hello'
titleCase('hello world'); // 'Hello World'
```

Capitalize strings in sentence or title case.

#36 Truncate with Ellipsis

```
const truncate = (str, max, suffix = '...') =>
str.length > max
? str.slice(0, max - suffix.length) + suffix
: str;

truncate('Hello World', 8); // 'Hello...'
truncate('Hello World', 5); // 'He...'
```

Truncate a string to a max length, appending suffix.

#37 Parse & Build Query Strings

```
// Parse
const parseQuery = str =>
Object.fromEntries(new URLSearchParams(str.replace(/^\?/, '')));

// Build
const buildQuery = obj =>
'?' + new URLSearchParams(obj).toString();

// Usage
parseQuery('?name=dev&tool=snippets');
// {name: 'dev', tool: 'snippets'}
buildQuery({page: 2, sort: 'asc'});
// '?page=2&sort=asc'
```

Convert between URL query strings and objects.

#38 Format Date

```
function fmtDate(date, locale = 'en-US', opts = {}) {
return new Date(date).toLocaleDateString(locale, {
weekday: 'short', year: 'numeric', month: 'short',
day: 'numeric', ...opts
});
}

fmtDate(new Date());
// 'Wed, Aug 13, 2025'
fmtDate(new Date(), 'en-US', { dateStyle: 'full' });
// 'Wednesday, August 13, 2025'
```

Format dates in localized, human-readable formats.

#39 Generate UUID v4

```
function uuid() {
return crypto.randomUUID
? crypto.randomUUID()
: ([1e7,-1e3,-4e3,-8e3,-1e11].join('-').replace(/[0184]/g, c =>
(c ^ crypto.getRandomValues(new Uint8Array(1))[0] % 16 >> c / 4).toString(16)[1] || 0
));
}

uuid(); // 'f5a9b2c3-1d4e-4f5a-9b3c-8e2d7a1c0b4f'
```

Generate a RFC 4122 compliant UUID v4.

Browser APIs

#40 Copy to Clipboard

```
async function copy(text) {
try {
await navigator.clipboard.writeText(text);
return true;
} catch {
const ta = document.createElement('textarea');
ta.value = text;
document.body.appendChild(ta);
ta.select();
const ok = document.execCommand('copy');
document.body.removeChild(ta);
return ok;
}
}

await copy('Hello clipboard!');
```

Copy text to the clipboard asynchronously.

#41 Download File

```
function download(content, name = 'file.txt', type = 'text/plain') {
const blob = new Blob([content], { type });
const url = URL.createObjectURL(blob);
const a = document.createElement('a');
a.href = url;
a.download = name;
a.click();
URL.revokeObjectURL(url);
}

download(JSON.stringify(data), 'data.json', 'application/json');
```

Trigger a file download from JavaScript.

#42 Toast Notification

```
function toast(msg, ms = 3000) {
const t = document.createElement('div');
Object.assign(t.style, {
position: 'fixed', bottom: '20px', right: '20px',
padding: '12px 20px', background: '#333', color: '#fff',
borderRadius: '8px', zIndex: 9999, fontSize: '14px'
});
t.textContent = msg;
document.body.appendChild(t);
setTimeout(() => t.remove(), ms);
}

toast('Saved successfully!');
```

Show a lightweight, self-removing notification.

#43 Detect Dark Mode

```
const isDark = () =>
window.matchMedia('(prefers-color-scheme: dark)').matches;

// Watch for changes
window.matchMedia('(prefers-color-scheme: dark)')
.addEventListener('change', e => {
document.body.classList.toggle('dark', e.matches);
});
```

Detect user preference and react to changes.

#44 Local Storage JSON Wrapper

```
const store = {
set(k, v) { localStorage.setItem(k, JSON.stringify(v)); },
get(k, d = null) {
try { return JSON.parse(localStorage.getItem(k) || 'null') ?? d; }
catch { return d; }
},
remove(k) { localStorage.removeItem(k); }
};

store.set('user', {name: 'Alice'});
const user = store.get('user'); // {name: 'Alice'}
```

JSON-safe localStorage that handles serialization.

#45 Cookie Manager

```
const cookie = {
set(name, val, days = 7) {
const d = new Date();
d.setTime(d.getTime() + days * 864e5);
document.cookie = `${name}=${encodeURIComponent(val)};expires=${d.toUTCString()};path=/`;
},
get(name) {
const m = document.cookie.match(new RegExp('(?:^|; )' + name.replace(/[.$?*|{}()\[\]\/\^\-]/g,
'\\$&') + '=([^;]*)'));
return m ? decodeURIComponent(m[1]) : undefined;
},
remove(name) { this.set(name, '', -1); }
};
```

Create, read, and delete cookies with expiration.

#46 Get URL Parameters

```
// All params
const params = Object.fromEntries(new URLSearchParams(window.location.search));

// Single param
const getParam = name =>
new URLSearchParams(window.location.search).get(name);

// Usage
const { page, sort } = params; // {page:'2', sort:'asc'}
const p = getParam('page'); // '2'
```

Extract query parameters from the current URL.

#47 Element in Viewport

```
function inViewport(el, threshold = 0) {
const r = el.getBoundingClientRect();
const vp = {
top: 0, left: 0,
bottom: window.innerHeight || document.documentElement.clientHeight,
right: window.innerWidth || document.documentElement.clientWidth
};
return (
r.bottom >= vp.top * (1 + threshold) &&
r.top <= vp.bottom * (1 - threshold) &&
r.right >= vp.left * (1 + threshold) &&
r.left <= vp.right * (1 - threshold)
);
}
```

Check if an element is visible within the viewport.

CSS & Layout

#48 CSS Gradient String Generator

```
function gradient(colors, angle = 90) {
return `linear-gradient(${angle}deg, ${colors.join(', ')})`;
}

gradient(['#ff0000', '#00ff00', '#0000ff'], 45);
// 'linear-gradient(45deg, #ff0000, #00ff00, #0000ff)'
```

Generate linear gradient CSS strings programmatically.

#49 Viewport Dimensions

```
function viewport() {
return {
width: window.innerWidth || document.documentElement.clientWidth,
height: window.innerHeight || document.documentElement.clientHeight,
scrollX: window.pageXOffset,
scrollY: window.pageYOffset
};
}

// Usage
const { width, height } = viewport();
console.log(`${width}x${height}`);
```

Get current viewport dimensions and scroll position.

#50 Format Currency

```
function fmtCurrency(amount, currency = 'USD', locale = 'en-US') {
return new Intl.NumberFormat(locale, {
style: 'currency',
currency
}).format(amount);
}

fmtCurrency(1234.56); // '$1,234.56'
fmtCurrency(99.99, 'EUR'); // '€99.99'
```

Format numbers as localized currency strings.

#51 Device Pixel Ratio

```
const isHiDPI = window.devicePixelRatio > 1;
const imageScale = window.devicePixelRatio || 1;

// Usage: choose image based on DPR
const img = isHiDPI ? 'image@2x.jpg' : 'image.jpg';
```

Detect high-DPI displays for image optimization.

Performance & Debugging

#52 Measure Execution Time

```
// Console timer
console.time('task');
await someAsyncOp();
console.timeEnd('task'); // task: 42.5ms

// Wrapper
async function timeIt(fn, label = 'Task') {
const t = performance.now();
const result = await fn();
console.log(`${label}: ${(performance.now() - t).toFixed(2)}ms`);
return result;
}

await timeIt(() => fetch('/api'), 'API Call');
```

Profile how long async and sync functions take.

#53 Lazy Load Images

```
function lazyLoad() {
const imgs = document.querySelectorAll('img[data-src]:not([src])');
const io = new IntersectionObserver((entries) => {
entries.forEach(e => {
if (e.isIntersecting) {
e.target.src = e.target.dataset.src;
io.unobserve(e.target);
}
});
}, { threshold: 0.1 });
imgs.forEach(img => io.observe(img));
}

lazyLoad();
```

Load images only when they enter the viewport.

#54 Memory Usage Helper

```
function sizeOf(obj) {
const bytes = JSON.stringify(obj).length;
const units = ['B', 'KB', 'MB', 'GB'];
let i = 0;
let size = bytes;
while (size >= 1024 && i < units.length - 1) {
size /= 1024; i++;
}
return `${size.toFixed(2)} ${units[i]}`;
}

sizeOf({a:1, b:[1,2,3]}); // '28 B'
```

Estimate memory usage of data structures.

#55 Assert Helper

```
function assert(condition, msg = 'Assertion failed') {
if (!condition) {
console.error(`Assert: ${msg}`, new Error().stack);
throw new Error(msg);
}
}

// Usage
assert(user.id, 'User must have an ID');
assert(typeof name === 'string', 'Name must be a string');
```

Runtime assertions for development and testing.

Unlock DevFlow Suite Pro

This is a free sample from the JavaScript Snippets Vault.

Upgrade to Pro for the full experience and 98 bonus snippets.

What is in Pro:

- All 150 snippets (52 in this guide + 98 bonus)
- Cloud sync across all devices
- Export to ZIP with individual .js files
- Team sharing & collaboration
- Custom themes & color palettes
- Priority email support

devflowsuite.com

$5/month • Cancel anytime